

DDA5001 Homework 1 — Answer Draft

The Chinese University of Hong Kong, Shenzhen

Student review and final wording are required. The AI-collaboration cases summarize this conversation's actual result-based follow-up exchanges.

Problem 1: Concepts and fundamental knowledge

(a) Supervised learning uses examples with target labels or values to learn a mapping from inputs to targets. Unsupervised learning uses inputs without such targets to discover structure, such as clusters or lower-dimensional representations.

(b) Statement 2 is true. When $n < d$, $\text{rank}(X) \leq n < d$, so $\ker(X)$ contains a nonzero vector. If $\hat{\theta}$ minimizes least squares, so does $\hat{\theta} + v$ for every $v \in \ker(X)$; there are infinitely many minimizers. Regression ordinarily fits numerical targets, a perceptron need not have a unique separating solution, and least squares is not an MLE for every noise distribution.

(c) For every nonzero $v \in \mathbb{R}^d$, full column rank means $Xv \neq 0$. Hence $v^\top X^\top X v = \|Xv\|_2^2 > 0$. Therefore $X^\top X$ is positive definite and invertible, giving a unique least-squares minimizer. The Hessian of $\|X\theta - y\|_2^2$ is $2X^\top X \succ 0$, so the objective is strongly convex.

Problem 2: Least squares without full column rank

(a) Write the supplied SVD as $X = V\Sigma_1 U_1^\top$ and complete U_1 with U_2 to an orthogonal basis of $\mathbb{R}^d$. Because Σ_1 is invertible, minimizing $\|V\Sigma_1 z - y\|_2^2$ over $z = U_1^\top \theta$ gives $z^* = \Sigma_1^{-1} V^\top y$. Thus all solutions are

$$\theta = U_1 \Sigma_1^{-1} V^\top y + U_2 w, \quad w \in \mathbb{R}^{d-n}.$$

Indeed $U_1^\top U_2 = 0$, and $U_2 w$ spans $\ker(X)$. Since $\text{rank}(X) = n$, X is onto $\mathbb{R}^n$; every listed solution satisfies $X\theta = y$, and the optimal value is 0.

(b) Differentiating the regularized objective yields $2X^\top(X\theta - y) + 2\lambda\theta = 0$. Since $v^\top(X^\top X + \lambda I)v = \|Xv\|_2^2 + \lambda\|v\|_2^2 > 0$ for nonzero v , the unique solution is

$$\theta_\lambda = (X^\top X + \lambda I)^{-1} X^\top y.$$

The inverse is a theoretical expression; a numerical implementation should solve the linear system.

Problem 3: Robust linear regression

(a) For residuals $r_i = (X\theta - y)_i$, the i.i.d. Laplace likelihood gives

$$-\log p(y \mid X, \theta) = n \log(2b) + \frac{1}{b} \sum_{i=1}^n |r_i|.$$

Both $n \log(2b)$ and the positive factor $1/b$ do not affect the minimizer in θ . The MLE therefore minimizes $\sum_i |(X\theta - y)_i|$.

(b) For the specified shifted smoothing,

$$h'_\mu(z) = \begin{cases} -1 & z \leq -\mu, \\ z/\mu & |z| < \mu, \\ 1 & z \geq \mu, \end{cases} \quad \nabla L(\theta) = X^\top g(X\theta - y),$$

where $X \in \mathbb{R}^{n \times d}$, $X\theta - y, g(X\theta - y) \in \mathbb{R}^n$, and $\nabla L(\theta) \in \mathbb{R}^d$. The branches agree at $\pm\mu$. Here $h_\mu(0) = \mu/2$; the implemented objective is a sum.

(c) The supplied $n = 1000, d = 50$ data were used with $\mu = 10^{-5}$, $\alpha = 0.001$, $T = 1000$, and `default_rng(0)`. Numerical least squares gave $\|\hat{\theta}_{\text{LS}} - \theta^*\|_2 = 59.51363648$. The Huber descent error changed from 8.98993999 at $k = 0$ to 5.61872016 at $k = 1000$; its sum loss changed from 256704.09771 to 252724.64378. The code loads θ^* only after fitting to calculate these errors. Figure 1 uses all 1001 recorded states; the objective and errors are in `regression_history.csv`.

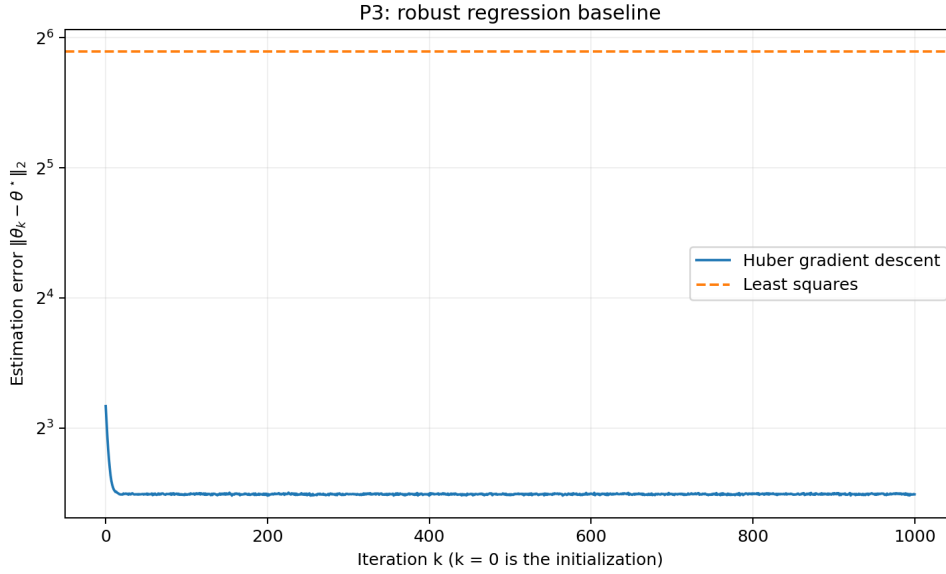


Figure 1: P3 parameter error versus update count, with the least-squares reference.

(d) The scalar check at $z/\mu = (-2, -0.5, 0, 0.5, 2)$ returned $h'_\mu(z) = (-1, -0.5, 0, 0.5, 1)$ for both tested values of μ . The small matrix gives $\nabla L = (0.5, 1.0)^\top$ and $\nabla L^\top v = 1.1$. Selected central differences are shown below; the full values are in `gradient_verification.csv`.

μ	t/μ	sum difference	mean difference	$ D_t L - 1.1 $
1	1	0.7625	0.1525	0.3375
1	0.1	1.1000	0.2200	1.33×10^{-15}
1	10^{-4}	1.100000000003	0.220000000001	2.77×10^{-12}
10^{-5}	1	0.7625	0.1525	0.3375
10^{-5}	0.1	1.1000	0.2200	6.66×10^{-16}
10^{-5}	10^{-4}	1.100000000002	0.220000000000	2.41×10^{-12}

The first-order Taylor formula $L(\theta + tv) = L(\theta) + t\nabla L(\theta)^\top v + o(t)$ explains convergence of the loss-change ratio to 1.1. At $t/\mu = 1$, the step is too large for a local approximation and crosses branches. Tiny steps can amplify subtraction roundoff; the supplied $10^{-4}\mu$ steps still give small absolute errors, so the table does not claim an observed failure at that size. The mean gradient is $(0.1, 0.2)^\top = \nabla L/5$ and its directional derivative is 0.22. To make $\theta - \alpha_{\text{sum}} \nabla L$ equal to a mean-gradient update, use $\alpha_{\text{mean}} = 5\alpha_{\text{sum}}$. The baseline code uses the sum gradient without division by n .

Problem 4: Perceptron convergence

Assume $\theta_0 = 0$, and index k by mistake updates. (a) Each of the finitely many values $y_i \theta^* x_i$ is strictly positive. Their minimum ρ is therefore positive.

(b) For the example selected at update k ,

$$\theta_k^\top \theta^* = \theta_{k-1}^\top \theta^* + y_{k-1} x_{k-1}^\top \theta^* \geq \theta_{k-1}^\top \theta^* + \rho.$$

Induction from $\theta_0 = 0$ yields $\theta_k^\top \theta^* \geq k\rho$.

(c) A mistake under the specified sign convention implies $y_{k-1} \theta_{k-1}^\top x_{k-1} \leq 0$ (equality can arise for a negative label). Consequently,

$$\|\theta_k\|_2^2 = \|\theta_{k-1}\|_2^2 + 2y_{k-1} \theta_{k-1}^\top x_{k-1} + \|x_{k-1}\|_2^2 \leq \|\theta_{k-1}\|_2^2 + \|x_{k-1}\|_2^2.$$

(d) Since each $\|x_i\|_2 \leq R$, induction gives $\|\theta_k\|_2^2 \leq kR^2$.

(e) For $k \geq 1$, combining (b) and (d) gives

$$\frac{\theta_k^\top \theta^*}{\|\theta_k\|_2} \geq \frac{\sqrt{k}\rho}{R}.$$

Cauchy–Schwarz bounds the left side above by $\|\theta^*\|_2$. Hence every possible mistake count obeys $k \leq R^2 \|\theta^*\|_2^2 / \rho^2$; the algorithm cannot continue making mistakes beyond this finite bound. This concerns mistake updates on strictly separable data, not epochs or a nonseparable sample.

Problem 5: Perceptron and pocket

(a) The features are mean intensity and mean absolute left–right asymmetry of each 16×16 image, with input $(1, \text{intensity}, \text{asymmetry})^\top$. The code predicts $+1$ at score zero. It updates the current weights by $\theta \leftarrow \theta + yx$ at a uniformly sampled current mistake. The pocket stores a separate copy only if the new training error is strictly smaller; ties retain the earlier pocket. Training never reads test labels. The run used seed 0 and 2000 mistake updates.

(b) The filtered data contain 1669 training and 434 test images. At update 2000, current train/test errors are 0.04014380/0.07142857 and pocket train/test errors are 0.03235470/0.06451613. Figure 2 and `trajectory.csv` show the full trajectory. Pocket training error equals the running minimum of current training error and had zero increases. Current training error fluctuated. The pocket test curve rose twice during this run even though its final value is below the current test error; no test monotonicity follows from selection on training error.

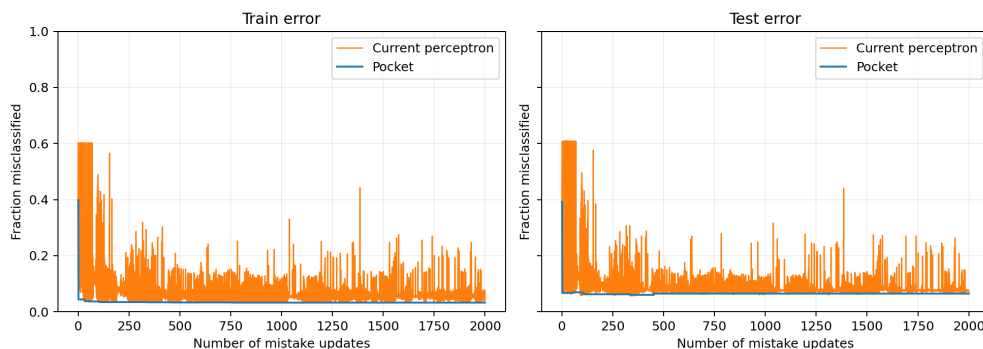


Figure 2: P5 current and pocket errors on training and held-out data.

(c) Figure 3 uses 500 training images from each class and overlays the two final boundaries. The bias weight is the intercept of the score $\theta_0 + \theta_1 \text{intensity} + \theta_2 \text{asymmetry}$, allowing a boundary away from the feature-space origin. The supplied plotting routine handles a vertical boundary when $\theta_2 = 0$ and a constant classifier when both feature weights are zero.

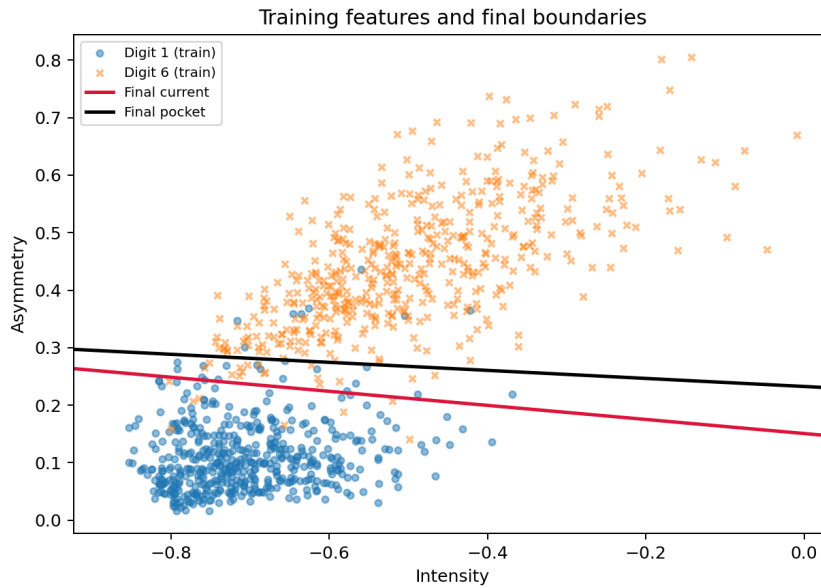


Figure 3: P5 training features and final current and pocket decision boundaries.

(d) Before running the check, we expected the pocket training error to be nonincreasing and equal the running minimum of current training errors. We did not expect this for test error. In the seven-row toy trace:

$$E_{\text{in}}(\theta_k) = (.50, .50, .25, .50, .25, .50, .25),$$

$$E_{\text{in}}(\hat{\theta}_k) = (.50, .50, .25, .25, .25, .25, .25).$$

At update 2, current weights $(0, -1, 0)$ predict $+1$ for the first three samples and -1 for the fourth, so only sample 2 is wrong: $E_{\text{in}} = 1/4$. Because this improves on $1/2$, the pocket copies these weights. At update 3, current weights $(-1, -1, 0)$ make two errors, so the pocket remains $(0, -1, 0)$. The audit compared every saved pocket error to the cumulative minimum of current errors and recomputed errors from saved weights. Copying matters because an in-place change to an aliased current array would silently change the stored pocket. A temporary pocket test-error rise alone is compatible with this training-only selection rule.

Reproduction notes

Source and numerical outputs are in the accompanying `src/` and `results/` directories. The original teacher files were preserved. The P5 run used equivalent `.npz` copies of the teacher `.mat` matrices because the available plotting Python lacked SciPy; conversion and input hashes are recorded in the work log.

Problem 3(e): AI collaboration case

Tool and prior expectation. I used Codex (GPT-6) to complete the mathematical TODOs in `src/p3.py` and interpret checks. Before any verification run, the conversation recorded this expectation: the sum-loss gradient should agree with a small directional finite difference; the mean-loss gradient should be the sum gradient divided by n ; too large or too small a step may enlarge numerical error. This was a prediction, not a measured result.

Actual result-based follow-up. After the run, I returned rows from `gradient_verification.csv`: at $\mu = t = 10^{-5}$ the central difference was 0.7624999999999995 versus analytic 1.1; at $t \approx 10^{-6}$ it was 1.0999999999999994, and the mean-loss difference was 0.22000000000000004. My follow-up asked why the larger step makes the directional finite difference depart from 1.1, and how this differs from the first-order Taylor loss-change error. The agent explained that a step of size μ crosses smoothing branches and averages slopes away from the current point. It distinguished $|D_t L - \nabla L^\top v|$ from $|L(\theta + tv) - L(\theta) - t \nabla L^\top v|$. At $t = \mu$ these were 0.3375 and 1.24×10^{-5} ; at $t = 0.1\mu$ they were approximately 6.66×10^{-16} and 1.32×10^{-7} . A separate follow-up checked `huber_branches.csv`: $h_\mu(0) = \mu/2$ and both outer branches matched the handout.

Decision. I retained the implementation. The code applies the handout's inner formula $z^2/(2\mu) + \mu/2$, outer formula $|z|$, and `np.sum`; its gradient returns $X^\top g$ without division by n . The small-example sum gradient is $(0.5, 1)^\top$, while the mean gradient is $(0.1, 0.2)^\top$. Their directional derivatives 1.1 and 0.22 and the small-step differences agree. The sum-to-mean difference ratio is 5, as expected for five samples, but that ratio alone follows from dividing one loss by five; branch values and code provide additional evidence. A mean-gradient implementation would need $\alpha_{\text{mean}} = 5\alpha_{\text{sum}}$ for the same small-example update. The original prompts and replies remain in this Codex conversation; full evidence is in the cited CSVs.

Problem 5(e): AI collaboration case

Tool and prior expectation. I used Codex (GPT-6) for the perceptron and pocket TODOs and their audit. Before verification, the conversation recorded that pocket training error should be the running minimum of current training error and hence nonincreasing; no such guarantee was predicted for test error.

Actual result-based follow-up. I returned the seven-row `toy_trace.csv` and asked the agent to recompute the update-2 error and check the strict pocket rule. Its reply checked the four predictions at weights $(0, -1, 0)$: the three inputs $(1, 0, 0)$ have score zero and predict $+1$; the fourth $(1, 1, 0)$ has score -1 and predicts -1 . Only the second label is wrong, so the error is $1/4$. At update 1 the current error tied the stored 0.5, so pocket stayed at zero; update 2 improved 0.5 to 0.25, so it copied the new weights; update 3 raised current error to 0.5, so pocket remained at 0.25. I later returned two rises from `trajectory.csv`: at update 54, pocket train/test errors changed $0.037148 \rightarrow 0.036549/0.066820 \rightarrow 0.069124$; at update 453, they changed $0.033553 \rightarrow 0.032954/0.059908 \rightarrow 0.064516$. The agent checked that each new current training error was strictly below the previous pocket error and that pocket copied the new weights.

Decision. I retained the code. Its comparison uses only training error, so rises in held-out error do not violate the pocket guarantee. In the toy trace the pocket sequence $(.5, .5, .25, .25, .25, .25, .25)$ equals the running minimum of current errors. The full trajectory audit checks this equality for every saved state. Test labels are loaded and used to score saved weights only after training finishes. Copying the pocket array prevents a later in-place current update from changing the stored best state. The original prompts and replies remain in this Codex conversation; the CSVs contain the numeric evidence.